

- **BiteFixes Enterprise Architecture Manual**

- **Volume 03**

- **Canonical Data Model (BCDM)**

- **Document ID:** BFA-003
- **Version:** 1.0
- **Status:** Draft
- **Language:** English
- **Architecture Level:** Enterprise
-

- **Table of Contents**

- 1. Introduction
- 2. Canonical Model Philosophy
- 3. Universal Entity Model
- 4. Company Domain
- 5. Customer Domain
- 6. Device Domain
- 7. Service Domain
- 8. Ticket Domain
- 9. Conversation Domain

10. Message Domain

11. Knowledge Domain

12. Technician Domain

13. Inventory Domain

14. Workflow Domain

15. Policy Domain

16. Event Domain

17. Audit Domain

18. Relationships

19. Data Ownership

20. Future Evolution

-

• Chapter 1

• Introduction

- The BiteFixes Canonical Data Model (BCDM) defines the official business entities used throughout the platform.
- It is **the single source of truth**.
- Everything else is generated from this model.
- Including:
 - PostgreSQL
 - FastAPI
 - Flutter

- Web Portal
- AI
- Reports
- Integrations
- Documentation
-

• Chapter 2

• Canonical Model Philosophy

- The canonical model represents the business.
- It does **not** represent:
- PostgreSQL tables
- Python classes
- JSON responses
- API payloads
- Instead, it defines the enterprise concepts.
-

• Chapter 3

• Universal Entity Model

- Every business entity follows the same structure.
- Business Entity

↓

Identity

↓

Ownership



Lifecycle



Relationships



Metadata



Audit

- Every entity inherits the following attributes.
-

- **Identity**

- InternalID

UUID

Code

Name

-

- **Ownership**

- CompanyID

CreatedBy

UpdatedBy

-

- **Lifecycle**

- Status

CreatedAt

UpdatedAt

DeletedAt

- Soft delete is mandatory.

-

- **Versioning**

- Version

Revision

SchemaVersion

- Future-proof design.

-

- **Metadata**

- Tags

Labels

Notes

Custom Fields

- Metadata must never replace official business fields.

-

- **Chapter 4**

- **Company**

- **Definition**

- Represents one independent organization using BiteFixes.
- One company owns all business information.
-

- **Responsibilities**

- Company is responsible for:
- Configuration
- Licensing
- AI Configuration
- Branding
- Languages
- Timezone
- Business Hours
- Integrations
-

- **Canonical Attributes**

- UUID

Legal Name

Trade Name

Slug

Tax ID

Country

State

City

Address

Postal Code

Phone

Email

Website

Business Type

Plan

Timezone

Currency

Default Language

Status

-

- **Relationships**

- Company

↓

Customers

↓

Devices

↓

Services

↓

Tickets

↓

Knowledge

↓

Users

↓

Inventory

↓

Reports

- Everything belongs to one company.
-

- **Chapter 5**

- **Customer**

- **Definition**

- Represents a commercial relationship.
- Not simply a person.
-

- **Canonical Attributes**

- UUID

CompanyID

Full Name

Document

Phone

Email

Address

Preferred Language

Preferred Contact

Customer Type

Status

Created At

Last Contact

-

- **Business Intelligence Attributes**

- Customer Score

Lifetime Value

Risk Level

Loyalty Level

Preferred Services

Average Repair Cost

Average Resolution Time

- These values are calculated.
- Never manually edited.
-

- **Relationships**

- Customer

↓

Devices

↓

Tickets

↓

Conversations



Invoices



Payments



Notifications

-

• Chapter 6

• Device

• Definition

- Represents a managed technological asset.
- Not only phones.
- Examples:
 - Smartphone
 - Notebook
 - Desktop
 - Printer
 - Server
 - Router
 - Switch
 - CCTV DVR
 - Tablet
 -

- **Canonical Attributes**

- UUID

CompanyID

CustomerID

Category

Brand

Model

Serial Number

IMEI

MAC Address

Operating System

Purchase Date

Warranty Expiration

Current Condition

Status

-

- **Calculated Attributes**

- Repair Count

Average Repair Cost

Last Repair Date

Warranty Claims

Failure Index

-

- **Relationships**

- Device

↓

Diagnostics

↓

Tickets

↓

Repairs

↓

Warranty

↓

Attachments

-

- **Chapter 7**

- **Service**

- **Definition**

- Represents a business capability offered by the company.
- Examples:
- Notebook Repair

Phone Repair

Network Installation

Server Maintenance

Data Recovery

Printer Repair

-

- **Canonical Attributes**

- UUID

CompanyID

CategoryID

Name

Description

Base Price

Estimated Time

Warranty Days

Status

-

- **AI Attributes**

- Intent

Knowledge Tags

Suggested Questions

Automation Level

-

- **Relationships**

- Service

↓

Knowledge

↓

Tickets

↓

Workflows

↓

Policies

-

- **Chapter 8**

- **Ticket**

- This is the central entity of the platform.
- Everything revolves around it.
-

- **Canonical Attributes**

- UUID

CompanyID

CustomerID

DeviceID

ServiceID

TechnicianID

ConversationID

Priority

Status

Diagnosis

Solution

Estimated Cost

Final Cost

Warranty

Created At

Closed At

-

- **Ticket Lifecycle**

- Draft

↓

Open

↓

Diagnosis

↓

Waiting Approval

↓

Waiting Parts

↓

Repair

↓

Testing

↓

Ready

↓

Delivered

↓

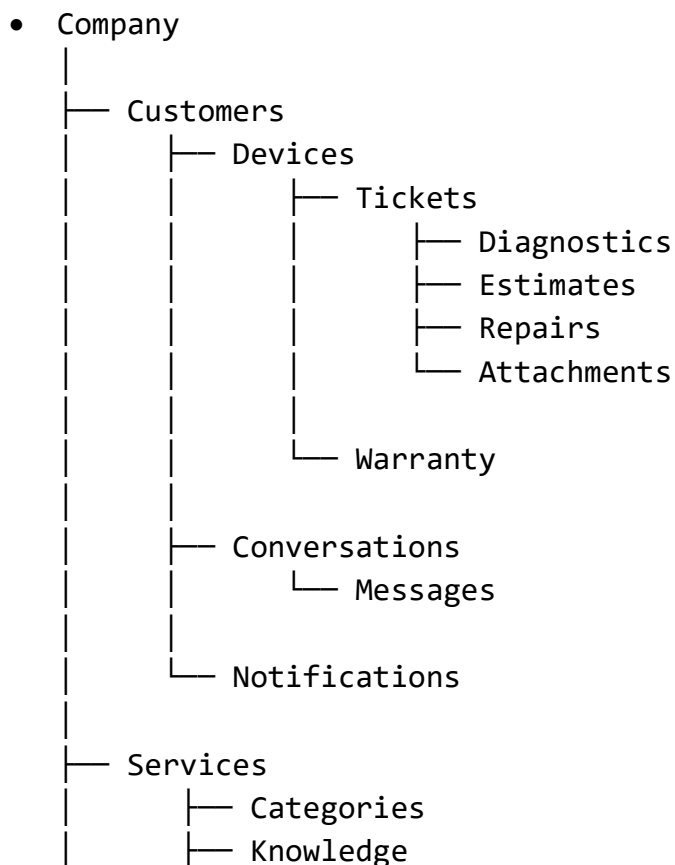
Closed

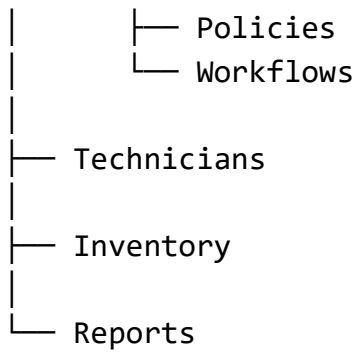
↓

Archived

- No state transitions outside this workflow are allowed unless explicitly defined by policy.
-

• Canonical Relationship Diagram





-

• Enterprise Rule

- One of the most important rules in BiteFixes is:
- **The Canonical Data Model is technology-independent.**
- That means:
- If tomorrow PostgreSQL is replaced by another database, the BCDM does not change.
- If FastAPI is replaced, the BCDM does not change.
- If Flutter is replaced by another framework, the BCDM does not change.
- The **business model remains stable**, while the technology can evolve independently.
-

• Architecture Maturity Levels

- The Canonical Data Model should support the evolution of BiteFixes through progressive maturity:

Level	Description
Level 1	Basic CRUD platform
Level 2	Workflow-driven business platform
Level 3	AI-assisted service platform
Level 4	Autonomous business platform
Level 5	Predictive enterprise platform

Level 6 Self-optimizing enterprise
platform

- Each new feature should move the platform toward a higher maturity level without breaking compatibility with the canonical model.
-

- **Recommendation**

- A partir del próximo volumen dejaría de describir entidades aisladas y empezaría a definir **cada dominio con precisión de ingeniería**, incluyendo:
 - Business responsibilities.
 - Entities.
 - Value Objects.
 - Aggregates.
 - Domain Events.
 - Business Rules.
 - Use Cases.
 - Repository Contracts.
 - Application Services.
 - Policies.
 - Workflows.
 - APIs.
 - Acceptance Criteria.
 - Secure
 - Observable
 - AI-Ready
 - Enterprise-Grade